

## §1 — Abstract

Training large language models at scale on distributed systems requires informed engineering decisions. The Gipfelsturm course project tasked us with exactly this: either minimizing evaluation loss under fixed wall-clock budgets or maximizing training throughput at given model scales on the **Alps Clariden** supercomputer. However, analogous to training an LLM for a realistic deployment setting, we addressed both challenges through three coordinated investigations using Megatron-LM and the Nemotron-ClimbMix dataset. First, we measure single-node 8B dense training throughput across a curated set of configurations, identifying bottlenecks in the canonical Megatron launcher through profiling traces and reporting per-lever throughput changes for FP8 hybrid precision, larger micro-batches with selective activation recomputation, tied embeddings, and FlashAttention via Transformer Engine. Second, we run a wall-clock-budgeted benchmark comparing dense and Mixture-of-Experts architectures at 30-, 60-, and 120-minute budgets, measuring final validation loss at each operating point. Third, we conduct an empirical scaling sweep characterizing MoE all-to-all communication overhead on Slingshot-11 as a function of  $(\text{nodes} \times \text{expert parallelism} \times \text{top-k} \times \text{num\_experts})$ . Combining all three studies, we plot the Pareto frontier on the (throughput, eval-loss) plane at our target scale. Our results expose which Megatron-LM flags actually move the throughput-loss frontier on Alps Clariden and yield concrete operating-point recommendations for each budget regime.

## §2 — Introduction (draft)

Training large language models on modern GPU clusters is a multi-axis engineering act of balance. A training framework must balance numerical precision (BF16, FP8) against numerical stability, micro-batch size against activation memory, dense parameter density against the routing cost of sparse Mixture-of-Experts architectures, and intra-node NVLink bandwidth against inter-node Slingshot bandwidth. Each individual axis has received attention in isolation in prior work: FlashAttention reduces attention's memory traffic [Dao et al. 2022, Shah et al. 2024], Transformer Engine exposes FP8 matmul on Hopper, ZeRO shards optimizer state across data-parallel ranks [Rajbhandari et al. 2020], and MoE architectures decouple model capacity from per-token compute [Shazeer et al. 2017, Fedus et al. 2021, Jiang et al. 2024] but looking at the config files Megatron LLM repo we see: The axes interact, and the operating point that maximizes throughput or minimizes training loss depends on their combination. The system we use in this project is the **Alps Clariden** (Grace-Hopper GH200 compute nodes), is recent that such public efficiency numbers for combing Megatron knobs for this specific hardware combination are scarce.

Based on the Gipfelsturm course project that defines two challenges. 1.) Minimize evaluation loss within fixed wall-clock budgets (30 min / 1 h / 2 h) on up to 8 nodes and 2.) the highest tok/s/GPU at specified model scales. We engage with both and structure our work as three coordinated investigations that share methodology (tuning flags), dataset (Nemotron-ClimbMix), tokenizer (GPT-2 BPE), and reproducibility (a launch patterns around the canonical Megatron-LM launcher).

First, we address Challenge 2 at the single-node 8B scale. Starting from the canonical Megatron launcher (BF16, MBS=2), we profile the baseline with `torch.profiler` and then add (in separate runs) FP8 hybrid precision through Transformer Engine, micro-batch size 4 with selective activation recomputation, tied input/output embeddings, and `--no-create-attention-mask-`

**in-data-loader** (relying on FlashAttention's implicit causal-mask handling). We isolate the effect of micro-batch scaling from precision via an additional BF16+MBS=4 control run.

Next, observing that throughput alone is incomplete, for example because a configuration that achieves higher tok/s but worse loss per training token may not be preferable, we address Challenge 1 with a wall-clock-budgeted benchmark comparing 9 configurations spanning BF16/FP8 precision, local-expert vs expert-parallel MoE, and dense vs sparse architectures at three budgets (30/60/120 min). The cell main matrix is run on 8 nodes throughout to fix the parallelism budget.

Third, we conduct an empirical scaling sweep characterizing MoE all-to-all-related throughput on Slingshot-11. The sweep spans a subset of  $(\text{nodes} \in \{1, 2, 4, 8\}) \times (\text{EP} \in \{1, 4, 8\}) \times (\text{top-k} \in \{1, 2, 4\}) \times (\text{num\_experts} \in \{8, 16, 32, 64\})$  plus one FP8-expert control point. We use W&B-logged per-step throughput as the primary measurement and derive a scaling efficiency curve.

Combining all three, we produce a **Pareto frontier on the (throughput, eval-loss) plane** identifying which operating point dominates which course-defined budget and characterize how the standard Megatron-LM knobs combine on a recent system and present an actionable map for picking among them under each of the course's two challenges.

## §3 — Background

We provide background on the systems and algorithms this report observes.

**3.1 — Hardware: Grace-Hopper GH200 and Slingshot-11.** Each Alps Clariden node hosts four GH200 superchips connected by NVLink. Each superchip pairs a Grace ARM CPU with a Hopper H200 GPU via NVLink-C2C, giving the GPU near-DRAM access to the CPU's LPDDR5. The GPU itself has 96 GB HBM3e at 4.0 TB/s. Inter-node communication uses Slingshot-11; the team's infrastructure benchmark measures ~340 GB/s NCCL bus bandwidth intra-node and ~93 GB/s inter-node.

**3.2 — Numerical precision on Hopper.** Hopper's tensor cores expose two FP8 formats (E4M3, more precision; E5M2, more range) and run them at a higher peak FLOP/s than BF16 (1980 vs 990 TFLOP/s dense, per the H200 datasheet). Training requires per-tensor scaling factors; the "hybrid" recipe uses E4M3 in the forward pass and E5M2 in the backward [Micikevicius et al. 2022]. Transformer Engine implements both with online amax tracking. Our FP8 runs use this hybrid recipe via TE.

**3.3 — Attention: FlashAttention.** Standard attention materializes a  $[\text{batch} \times \text{heads} \times \text{seq} \times \text{seq}]$  matrix in HBM. FlashAttention restructures the computation as tiled, IO-aware matmul + incremental softmax, keeping the intermediate in SRAM [Dao 2022, Dao 2023]. FlashAttention-3 [Shah et al. 2024] adds Hopper-specific WGMMMA and TMA support. Transformer Engine selects an attention backend automatically based on shapes and dtypes; we identify which backend was active from our profile trace rather than choosing one manually.

**3.4 — Distributed training paradigms.** Common memory-sharding regimes are **DDP** (full replication), **ZeRO-1/2/3** [Rajbhandari et al. 2020] (progressive sharding of optimizer state,

gradients, parameters), and PyTorch's **FSDP/FSDPv2** [Zhao et al. 2023] (a native analogous implementation). Megatron-LM's `--use-distributed-optimizer` is functionally equivalent to ZeRO-1 (optimizer-state sharding only), with `--overlap-grad-reduce` / `--overlap-param-gather` overlapping communication with backward compute. All our runs use this Megatron path. We do not benchmark against the alternatives.

**3.5 — Mixture of Experts.** A standard transformer's feed-forward block is replaced with `N` parallel "experts" plus a router that selects `top-k` of them per token [Shazeer et al. 2017, Lepikhin et al. 2020, Fedus et al. 2021]. **Expert parallelism (EP)** shards experts across ranks, requiring an all-to-all token dispatch each layer. EP scales the model's total parameter count without proportionally scaling per-token compute, at the cost of inter-rank communication. Our §6 sweep operates in this regime.

## §4 — Methodology

### TODO.

- **4.1 — Hardware and container.** 8 nodes of Alps Clariden, GH200, container image `ngc-pytorch:26.01-py3-alps3` configured via EDF `~/edf/alps3.toml`.
- **4.2 — Software stack.** Megatron-LM
- **4.3 — Dataset and tokenizer.** Nemotron-ClimbMix subset `climbmix_small_megatron`, pre-tokenized with GPT-2 BPE (50,257-vocab). Same dataset/tokenizer for all experiments to enable comparison.
- **4.4 — Reproducibility tooling.** Sibling-launcher pattern: `launch.sh` (canonical) + `launch_profile.sh`, `launch_bf16_mbs4.sh`, `launch_stacked.sh`, `launch_stacked_3b.sh`, `launch_moe_sweep.sh`, `moe_sweep_driver.sh`. Each launcher writes a self-contained `.sbatch` file in `logs/`.
- **4.5 — Measurement protocol.** `tokens-per-sec-per-gpu` averaged over W&B-logged steps 20–49 (post-warmup steady state). Eval loss is the last logged `lm loss validation` for completed runs.

## §5 — Single-node dense throughput (draft)

**5.1 — Baseline and bottleneck identification.** We begin from the unmodified `launch.sh throughput 8b 50 1` configuration (BF16 mixed precision, micro-batch 2, global batch 256, sequence length 4096, grouped-query attention with 8 KV heads, Megatron distributed optimizer with `--overlap-grad-reduce` and `--overlap-param-gather`, FlashAttention via Transformer Engine, single node, 4 GH200 GPUs). A 50-step profiling run (`launch_profile.sh`) captures `torch.profiler` traces at steps 10–12 on rank 0, after the 10-step LR warmup and after autotuning caches have stabilized.

Figure F1 shows the chrome-trace decomposition for a representative step: `[X1]` % matmul kernels, `[X2]` % attention kernels (specific FA backend selected by TE: `[FA_version]`), `[X3]` % NCCL gradient reduction, `[X4]` % outside CUDA kernels. We do not assert the bottleneck identity in advance — the diagnosis is read off the trace once the profile job completes.

Headline baseline throughput: `[X tok/s/GPU]` averaged over steps 20–49 (W&B run `[ID]`).

**5.2 — Single-knob ablations (prior team work).** A team member previously ran single-knob ablations on an 8B FP8 hybrid baseline at micro-batch 1 (`gipfelsturm` and `single_node` W&B projects, 11 runs total). Of the 4 nominal replicates that ran to completion, the W&B `tokens-per-sec-per-gpu` values were 15,346 / 14,296 / 13,809 / 13,621 (mean 14,268, ~12% spread). One additional run (`2279409`) recorded 9,728, which we treat as an outlier of unknown origin.

Against this noise floor, the tested single-knob variants returned no clear effect: `--cuda-graph-impl local` and FP8 tensorwise vs delayed scaling were each within the replicate spread; TP=2 produced 13,150 tok/s/GPU, slightly below the baseline mean but inside the noise band. **The single-knob ablations yield a noise-band result for each lever rather than a positive identification of a winning knob.** Whether the bottleneck is kernel scheduling, memory bandwidth, or something else cannot be concluded from these data alone; we read it off the profile trace in §5.1.

**5.3 — Stacked optimizations.** Guided by the profile in §5.1, we ran a stacked experiment (`launch_stacked.sh`) combining five known levers: FP8 hybrid precision, micro-batch size 4 (with selective activation recomputation), tied input/output embeddings, and `--no-create-attention-mask-in-dataloader`. Compared to the unmodified baseline:

| Configuration                                                             | tok/s/GPU | Speedup vs baseline    |
|---------------------------------------------------------------------------|-----------|------------------------|
| <code>launch.sh</code> baseline (BF16, MBS=2)                             | [X]       | 1.0× (reference)       |
| Prior FP8 baseline mean (MBS=1, prior work)                               | 14,268    | [X_FP8] ×              |
| <code>launch_bf16_mbs4.sh</code> (BF16, MBS=4 + selective recompute)      | [X_MBS]   | [X_MBS] ×              |
| <code>launch_stacked.sh</code> (FP8 + MBS=4 + recompute + tied + no-mask) | [X_STACK] | [X_STACK] ×            |
| <code>launch_stacked_3b.sh</code> (same stack on 3B backbone, MBS=8)      | [X_3B]    | (different model size) |

Multiplicative decomposition (Figure F2): FP8-only `[A×]`, MBS-only `[B×]`, stacked `[C×]`. The composition bonus  $C / (A \times B) = [D]$  indicates whether the levers reinforce ( $>1$ ) or interfere ( $<1$ ).

**5.4 — Achieved compute vs theoretical ceiling.** For each configuration we estimate achieved TFLOP/s as  $6 \times N_{\text{active\_params}} \times \text{tok\_per\_sec\_per\_gpu}$ , the standard `6N` approximation; this excludes attention's `seq2` term so the resulting TFLOP/s is a lower bound. Figure F3 plots achieved TFLOP/s with the GH200's BF16 (990 TFLOP/s dense) and FP8 (1980 TFLOP/s dense) ceilings overlaid as horizontal lines. The stacked run reaches `[Y]%` of FP8 peak. A full roofline plot with arithmetic intensity on the x-axis would require profile-derived FLOP and HBM-byte counts we did not collect; we note that as future work.

*Bridge to §6.* At single-node we approach the FP8 ceiling for our MBS regime, but the course explicitly motivates scaling. At larger total capacity, dense models are forced into pipeline parallelism with its structural bubble cost; this motivates the MoE study that follows.

## §6 — Multi-node MoE scaling (draft)

**6.1 — Why all-to-all matters.** MoE layers replace the dense FFN with  $N$  parallel experts plus a router that sends each token to  $\text{top-}k$  of them. When experts are sharded across ranks ( $\text{EP} > 1$ ), each layer dispatches tokens via an all-to-all permutation, computes the experts, then gathers via a second all-to-all. The team's infrastructure benchmark on Alps Clariden reports  $\sim 340$  GB/s NCCL bus bandwidth intra-node (NVLink) vs  $\sim 93$  GB/s inter-node (Slingshot-11) — a  $\sim 3.6\times$  drop once EP spans nodes. We measure the throughput effect directly via the sweep below; we do not claim novel comm-overhead numbers without a literature comparison.

**6.2 — Sweep design.** We ran 14 cells with a 760M-active backbone,  $\text{MBS}=4$ , BF16 base precision, and Megatron's standard MoE recipe (FP32 router, softmax routing, aux-loss coefficient  $1e-2$ , router z-loss  $1e-3$ , grouped GEMM, alltoall dispatcher). Three axes vary:

- **Nodes  $\times$  EP** (8 cells):  $\text{nodes} \in \{1, 2, 4, 8\} \times \text{EP} \in \{1, 4, 8\}$  (subset), fixed  $\text{top-}k=2$ ,  $\text{num\_experts}=8$ . Tests the scaling efficiency frontier.
- **Top-k slice** (2 cells):  $\text{top-}k \in \{1, 4\}$  at  $\text{nodes}=8$ ,  $\text{EP}=8$ ,  $\text{num\_experts}=8$ . Doubles or quadruples per-token dispatch volume.
- **Expert-count slice** (3 cells):  $\text{num\_experts} \in \{16, 32, 64\}$  at  $\text{nodes}=8$ ,  $\text{EP}=8$ ,  $\text{top-}k=2$ . Note that `--moe-ffn-hidden-size` defaults to the dense FFN width when unset (Megatron `arguments.py:1016`), so all experts are full-width; increasing  $\text{num\_experts}$  therefore increases *total* parameters but keeps active compute per token constant. **This is not equivalent to DeepSeekMoE's setup**, which shrinks expert width when increasing expert count — we make no claim about sample efficiency from this slice.
- **Precision control** (1 cell): cell 8 setup with FP8 experts.

Each cell is one SLURM submission via `launch_moe_sweep.sh`, driven by `moe_sweep_driver.sh`. A cell with  $\text{num\_experts}=4$ ,  $\text{EP}=8$  was planned but dropped because Megatron asserts `num_experts % EP == 0`.

**6.3 — Strong-scaling efficiency.** Figure F4 plots  $T(\text{nodes}=1) / (T(\text{nodes}) \times \text{nodes})$  vs node count, one curve per EP setting, computed from W&B per-step throughput. Local-experts MoE ( $\text{EP}=1$ ) reaches `[E_LOCAL]`% efficiency at 8 nodes;  $\text{EP}=4$  across 4 nodes — first cell where the all-to-all spans the inter-node fabric — reaches `[E_EP4]`%;  $\text{EP}=8$  across 8 nodes reaches `[E_EP8]`%. We attribute the  $\text{EP}=1$  vs  $\text{EP}>1$  gap at fixed node count to additional all-to-all communication, but we do not separately measure NCCL kernel time and so cannot quantify the breakdown without a deeper profile pass.

**6.4 — Throughput vs sweep axes.** Figure F5 plots tok/s/GPU per cell. Doubling  $\text{top-}k$  from 1 to 2 to 4 (cells 9, 8, 10) changes throughput by `[...]`. Varying  $\text{num\_experts}$  at fixed EP and  $\text{top-}k$  (cells 8, 11, 12, 13) changes throughput by `[...]`. The FP8-experts control (cell 14) achieves `[...]` vs cell 8's BF16. Whether these patterns are bandwidth- or latency-bound at the all-to-all kernel level is not directly identifiable from W&B step timings; we describe the patterns and note an nsys-attributed breakdown as the natural follow-up.

**6.5 — Interpretation.** A throughput drop as  $\text{num\_experts}$  grows is consistent with per-expert kernel-launch overhead (smaller per-expert batches once tokens are split across more experts); roughly flat throughput is consistent with grouped-GEMM amortizing those overheads. We report what we observe and offer the corresponding interpretation.



*Bridge to §7.* Throughput alone cannot adjudicate between architectures of different sizes or sparsity levels. The practical question depends on (throughput, quality) — the wall-clock benchmark fills the eval-loss axis.

## §7 — Throughput–loss frontier (draft)

**7.1 — Combining throughput and eval-loss data.** The proposal positioned throughput as the primary metric. In practice, throughput in isolation cannot adjudicate between architectures of different sizes or sparsity levels. We therefore present the project's headline figure as a **joint frontier on the (throughput, eval-loss) plane**, combining throughput measurements from §§5–6 with eval-loss measurements from the wall-clock-budgeted benchmark.

The wall-clock benchmark (full protocol in `protocol.md`) trains 9 dense-vs-MoE configurations at three scales — small (30 min effective training), medium (60 min), large (120 min) — on 8 nodes, same dataset, tokenizer, and final-eval procedure. Of 27 main-matrix jobs plus 4 capacity add-ons: `[N_FINISHED]` reached `iteration end` cleanly; `[N_RECOVERED]` ran to the wall-clock budget but failed at the final checkpoint-save step (see §8). For the latter we recover the last logged `lm loss validation` from W&B history as the eval-loss proxy, with the caveat noted in §10.

**7.2 — Headline figure.** Figure F6 plots each surviving configuration as a point on the (tok/s/GPU, final eval loss) plane, colored by family (dense BF16, dense FP8, MoE local, MoE EP). The Pareto frontier is highlighted.

At the 30-min budget, `[CONFIG_S]` is on the frontier. At 60-min, `[CONFIG_M]`. At 120-min, `[CONFIG_L]`. No single configuration is Pareto-optimal across all budgets; `[1–2 sentence explanation referencing model size, Chinchilla-style scaling, and the MoE comm-overhead crossover from §6]`.

**7.3 — Engineering guidance.** Each per-budget recommendation is conditional on the actual data placement:

1. **Short-budget ( $\leq 30$  min):** if the 3B stacked configuration is on the frontier at this budget, it is the recommended operating point (Chinchilla-style: more tokens for the smaller model). If it is dominated, drop the recommendation.
2. **Medium-budget ( $\sim 1$  hour):** depends on whether any MoE configuration reaches lower eval loss than all four dense baselines at the same wall-clock; this is the MoE-win criterion defined in `protocol.md`.
3. **Long-budget / large-capacity ( $\sim 2$  hours,  $\geq 45$ B total params):** dense in this size class requires pipeline parallelism with a bubble cost; whether EP-MoE wins depends on whether the bubble cost exceeds the all-to-all cost we measure in §6. We do not claim this without numbers on both sides.

Recommendations are specific to Alps Clariden (GH200 + Slingshot-11), Nemotron-ClimbMix, Megatron-LM `core_v0.16.1`, container image `ngc-pytorch:26.01-py3-alps3`. We do not extrapolate beyond this configuration.

*Bridge to §8.* The same data also informs how future teams should set up runs.

## §8 — Operational findings: checkpointing (draft)

**8.1 — Empirical failure mode.** Of the 43 multi-node MoE runs in `wallclock-moe-scaling-benchmark`, 2 finished cleanly. A further 11 ran for the full intended wall-clock budget (runtimes match 1530 s / 3020 s / 7200 s for small / medium / large) and reached training losses in the expected range (2.16–3.15), but their W&B `state` is `crashed` because the job died during the final checkpoint-save step. This is consistent with a known Megatron-LM checkpoint-save issue on GH200/ARM64 noted in the project root README. The remaining runs are launcher-init failures (17 runs < 60 s, traced to user-side config errors unrelated to the checkpoint issue) and mid-training failures (9 runs).

**8.2 — Data recovery.** For the 11 "completed-but-uncheckpointed" runs we extract the most recent `lm loss validation` reading from W&B history via `wandb.Api.scan_history()` and treat it as the eval-loss proxy. We did not refine further (e.g. averaging multiple late readings); the recovered value is what appears in §7's Pareto table with the caveat in §10.

**8.3 — What we did not do, but flag as the most useful future work.** The course's "Possible Project Features" slide lists auto-resubmission, graceful timeout exits, and Torch Fault Tolerance / DiLoCo. **We implemented none of these.** What would have helped this project most concretely:

1. A SLURM signal handler catching `SIGUSR1` (`#SBATCH --signal=USR1@300`) and writing a checkpoint before the wall-clock limit, independent of the failing final-checkpoint path.
2. A re-submission script that resumes from the partial checkpoint via `--dependency=afterany:<jobid>`.
3. Integration with the distributed fault-tolerance APIs cited in the course slides.

## §9 — Discussion (stub)

**TODO.** Three honest points, expanded once §§5–8 numbers exist:

1. **Throughput is not the whole story.** The stacked dense run improved over the prior FP8 baseline by `[D×]`, but the 3B variant at the same stack achieves higher tok/s and worse eval loss. The proposal's "throughput primary, loss secondary" framing is too simple — they are a frontier.
2. **Multi-node MoE is comm-related-bound on Slingshot at the EP=8 / 8-node operating point.** Cells [...] show `[X%]` strong-scaling efficiency. Hierarchical all-to-all (DeepEP-style) is the obvious next lever but did not fit in the project timeline.
3. **Most of our wall-clock MoE benchmark runs crashed at final checkpoint.** This is a methodology finding about Megatron's checkpoint behavior on GH200, not about the experiment design. The recovered eval losses from W&B history suggest the design was sound; future iterations should add a graceful-exit checkpoint handler (§8.3).

## §10 — Threats to validity (stub)

**TODO.** Items to cover, in decreasing order of impact:

- **Recovered-eval-loss proxy** (§7, §8). For 11/13 surviving MoE runs we use a mid-training W&B `lm loss validation` reading, not a post-training final eval. Reported with explicit per-row marking in the Pareto table.

- **Single-seed runs throughout.** No statistical replication. We use the ~12% noise floor observed in Mykhaylo's 4 nominal replicates as an informal variance estimate.
  - **Cluster sharing.** Slingshot inter-node and capstor/iopsstor I/O are shared resources; some run-to-run variation is attributable to other users' load.
  - **6N approximation lower-bounds achieved TFLOP/s** in §5.4 (Figure F3).
  - **No nsys trace.** Comm-overhead in §6 is inferred from W&B step timings, not directly attributed to NCCL kernels.
  - **GPU-hour budget usage:** [X] of 64 budgeted.
- 

## §11 — Conclusion (stub)

**TODO.** One paragraph. Restate the three findings as engineering guidance for Alps users:

1. Single-node 8B dense reaches [Y%] of FP8 peak after the stack of FP8 + MBS=4 + selective recompute + tied embeds + no-mask. Per-lever breakdown in F2.
  2. Multi-node MoE at EP=8 across 8 nodes loses [X%] strong-scaling efficiency to comm. Per-axis numbers in F4–F5.
  3. Joint frontier (F6) identifies [CONFIG\_S] / [CONFIG\_M] / [CONFIG\_L] as the per-budget operating points for our scale, hardware, and dataset.
- 

## Appendix A — Crashed-run inventory (pointer)

See [course-project/wandb\\_exports/RUNS\\_SUMMARY.md](#) for the full inventory: 43 runs categorized into finished / launcher-failed / completed-but-uncheckpointed / mid-training-failed / capacity-probe-failed. The recovery method for the "completed but uncheckpointed" category is described in §8.2.

---

## Appendix B — Exact Megatron-LM flags per experiment (stub)

**TODO.** For each launcher we used, paste the exact CLI invocation Megatron actually saw. The launchers in the repo root ([launch.sh](#), [launch\\_profile.sh](#), [launch\\_bf16\\_mbs4.sh](#), [launch\\_stacked.sh](#), [launch\\_stacked\\_3b.sh](#), [launch\\_moe\\_sweep.sh](#)) generate the [.sbatch](#) scripts in [logs/](#); those [.sbatch](#) scripts are the ground truth.

---